MKM-ModelRisk

Integration Guide

How the platform works, how to restyle it, how the coding rules are enforced, how to move it to another SQL engine or a container, and how it takes identity from your SSO.

MKM Research Labs

Version 1.0 — August 23, 2026

Written against `main` at commit `642d4f6` (2026-08-23)

CONFIDENTIAL — PROPRIETARY

Legal notice

PROPRIETARY AND CONFIDENTIAL

This document describes the MKM-ModelRisk platform. The software itself is MIT-licensed (see `LICENSE` in the repository); this notice covers the contents of this document, which constitute the confidential material of MKM Research Labs ("MKM").

Any usage, reproduction, distribution, modification, or disclosure of this document or any portion thereof, without the express prior written authorisation from MKM Research Labs is strictly prohibited.

The document is provided on an "as is" basis. MKM Research Labs makes no representations or warranties of any kind, express or implied, regarding its accuracy, reliability, or suitability for any particular purpose.

All rights reserved. © 2022–2026 MKM Research Labs.

Governed by the laws of the United Kingdom.

Contents

1 About this guide **3**
1.1 Who it is for . . . 3
1.2 What it covers . . . 3
1.3 How to read the claims in it . . . 3

2 How the technology works **4**
2.1 One paragraph . . . 4
2.2 The shape of a request . . . 4
2.3 The event store . . . 5
2.4 Migrations . . . 5
2.5 Entry points and startup . . . 5
2.6 Where configuration comes from . . . 6
2.7 The console and the documents . . . 7

3 Changing the style **8**
3.1 The rule this section is about . . . 8
3.2 Where the values live . . . 8
3.3 How a token reaches the screen . . . 9
3.4 How a token reaches the PDFs . . . 9
3.5 Rebranding: the procedure . . . 10
3.6 What is not built yet . . . 10

4 The coding rules R1–R10 **11**
4.1 Why an integrator should care . . . 11
4.2 The rules . . . 11
4.3 Running the audit . . . 12
4.4 The reports . . . 12
4.5 Where the thresholds live . . . 13
4.6 The baseline . . . 13

5 Switching to a different SQL server **14**
5.1 Two different questions . . . 14
5.2 What is engine-specific, and where . . . 14
5.3 A port, step by step . . . 15
5.4 Immutability does not cross the border . . . 15

6 Running it inside a container **17**
6.1 Status: not shipped . . . 17
6.2 What has to be in the image . . . 17
6.3 A worked Dockerfile . . . 17
6.4 A worked Compose file . . . 18
6.5 The things that only bite once it is in a container . . . 19

7 Permissions and SSO **21**
7.1 The model in one sentence . . . 21
7.2 What arrives, and what happens to it . . . 21
7.3 Who is allowed to author a change . . . 21
7.4 The trust boundary — the part to get right . . . 22
7.5 Wiring it to an identity provider . . . 22
7.6 What is designed but not built . . . 23

A Environment variable reference **25**

B Verification checklist **26**

C Where things live **27**

1 About this guide

1.1 Who it is for

This guide is for the engineer who has to *take MKM-ModelRisk into an institution*: stand it up, point it at that institution's database, put its own brand on it, wire it to its single sign-on, and satisfy whoever asks how the thing is controlled. It assumes you can read Python and SQL. It does not assume you know model-risk governance — for what the screens *mean*, read `docs/user_guide.md`; this guide is about the machinery underneath them.

1.2 What it covers

§2. How the technology works — the event store, the read models, the request path, and where configuration comes from.
§3. Changing the style — the single file that holds every colour, size and typeface, how it reaches the browser and the PDFs, and how to rebrand safely.
§4. The coding rules R1–R10 — what each one is for, which are mechanically gated, and how to run the audit.
§5. Switching to a different SQL server — what is engine-specific, what is not, and the checklist for a port.
§6. Running it inside a container — what has to be in the image, a worked `Dockerfile` and Compose file, and the traps that only appear once the app is not on a developer's laptop.
§7. Permissions and SSO — who is allowed to author a change, where the trust boundary sits, and how to put an identity provider in front of it.

1.3 How to read the claims in it

Every statement here is written against the code at the commit on the title page, and the guide is explicit about the difference between three things that are easy to conflate:

- **Built and enforced** — there is code, and something fails if you break it.
- **Built, not enforced** — there is code, and nothing stops you bypassing it.
- **Designed, not built** — there is a design document and no code.

Where a section describes work you would have to do yourself (the SQL Server port, the container image), it says so plainly rather than presenting a plan as a feature.

2 How the technology works

2.1 One paragraph

MKM-ModelRisk is a model-risk-governance platform: it holds an institution's inventory of quantitative models, the validation and committee process around them, and the evidence trail that process produces. Its distinguishing property is that **the audit trail is the system of record**. Nothing is stored as a mutable row that somebody edited; every change is an *event* appended to a hash-chained log, and every screen you see is a projection rebuilt from that log. The application is a Flask app serving a dependency-free vanilla-JavaScript console, on PostgreSQL, with PDFs rendered through LaTeX.

2.2 The shape of a request

```
browser (vanilla JS console)
      |  fetch /api/...  + X-Auth-Request-User header
      v
Flask blueprint  (src/web/api_*.py)
      |  context_from_request(): resolve_principal -> Principal
      v
dispatch()       (src/command/dispatch.py)
      |  1. confirm-to-amend gate   2. authorize()   3. actor + reason
      v
domain command   (src/governance/**)
      |  append_event(...)                    read_view(...)
      v                                            ^
src/database/    ---- events (append-only) --------+
      |                      |
      |                      +--> projections update the *_view read models
      v
PostgreSQL
```

Four things about that path are worth understanding before you change anything.

Every mutation goes through one seam. `dispatch()` is the only sanctioned way to run a command that changes state. It enforces confirm-to-amend (the UI's confirmation dialog cannot be bypassed by calling the API directly), calls `authorize()`, and threads the resolved actor and the change reason into the handler, which stamps them on the event. A new endpoint that forgets to ask about permissions is not possible so long as it dispatches; the few paths that cannot dispatch — multipart uploads, which call their command directly — apply the same check through `request_actor()` in `src/web/_helpers.py`.

Every persistence call goes through one package. No module outside `src/database/` may contain SQL, a cursor, a connection or a driver import. That is coding rule **R6**, it is audited with zero tolerance, and it is the single reason §5 (moving to another SQL engine) is a bounded job rather than an open-ended one.

The log is append-only, and the database says so. Migration `0002_roles_grants.sql` grants the runtime role `SELECT` and `INSERT` on `events` and revokes `UPDATE`, `DELETE` and `TRUNCATE`. Immutability is therefore a property of the deployment's grants, not a convention in the application. Each event also carries `prev_hash` and `event_hash` — a SHA-256 chain — so tampering by a more privileged account is *detectable* even where it is not *prevented*.

Read models are caches, and are rebuilt on boot. The `*_view` tables are derived from the log and maintained incrementally. If one drifts — after a restore, or a data-directory move,

where no command ran to refresh it — records render blank even though the log is intact. This bit in production on 2026-07-29: an MRC meeting looked deleted when only its view had gone stale. Startup therefore replays every projection, so a stale cache cannot outlive a restart. Set `REBUILD_VIEWS_ON_STARTUP=0` to skip it if the estate ever grows large enough that a full replay per boot is undesirable.

2.3 The event store

The schema is deliberately small. `src/database/migrations/0001_event_store.sql`:

```
CREATE TABLE IF NOT EXISTS events (
    global_seq     BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    event_id       UUID NOT NULL DEFAULT gen_random_uuid(),
    aggregate_type TEXT NOT NULL,
    aggregate_id   TEXT NOT NULL,
    stream_seq     INTEGER NOT NULL CHECK (stream_seq > 0),
    event_type     TEXT NOT NULL,
    payload        JSONB NOT NULL DEFAULT '{}'::jsonb,
    actor          TEXT NOT NULL,
    reason         TEXT,
    recorded_at    TIMESTAMPTZ NOT NULL,
    prev_hash      TEXT,
    event_hash     TEXT NOT NULL,
    CONSTRAINT events_stream_unique UNIQUE (aggregate_type, aggregate_id, stream_seq)
);
```

A *stream* is one governed subject — a model, a product, a committee meeting — keyed by `(aggregate_type, aggregate_id)`. `stream_seq` is a gap-free 1-based counter within it, and the unique constraint is the optimistic-concurrency backstop: two writers cannot both claim position *n*. `global_seq` orders the whole log, and each projection records how far it has replayed in `projection_checkpoints`.

The public API of `src/database` is intent-named — `append_event`, `read_stream`, `read_view`, `replay`, `run_migrations` — and there is no escape hatch. If a query you need is not exposed, the correct move is to add a named function to that package, not to open a connection where you are.

2.4 Migrations

Numbered plain-SQL files under `src/database/migrations/`, applied in lexical order by the owner role and recorded in `schema_migrations`, so re-running is a no-op. Two details matter operationally:

- **The runtime role is substituted, not hardcoded.** The grants are written against `{{app_user}}` / `{{app_password}}`, which the runner fills in from `DB_APP_USER` / `DB_APP_PASSWORD`. A deployment that renames the role still gets grants against the role it actually connects as. The name is validated against a plain-identifier pattern first, because a role name cannot be passed as a query parameter.
- **A PostgreSQL advisory lock is held while migrating.** Under gunicorn several processes may start at once; without the lock two of them read the same migration as pending, both execute its DDL, and the loser dies on the `schema_migrations` primary key — a worker that crashes on boot with nothing in the log to explain it.

2.5 Entry points and startup

Command	What it is
`python model.py`	Development server. Werkzeug, single process, auto-reloading.
`gunicorn wsgi:application`	The real thing. Settings come from `gunicorn.conf.py`, which is a thin wiring of `config/web.py`.
`scripts/start.sh`	One-command setup on Linux/macOS: toolchain checks, venv, dependencies, local cluster, then launch. Idempotent.
`scripts/dev_app.sh`	The verification app on port 5016 against the scratch database. Refuses to start if the resolved database is a protected one.

The one-time startup work — migrations, seeding the admin reference lists, and the read-model rebuild — is `bootstrap()` in `src/web/_startup.py`. Under gunicorn it runs in the **master**, in the `on_starting` hook, before any worker is forked; the master sets `MODELRISK_BOOTSTRAP_DONE` in the environment and the forked workers inherit it and skip the work. Under any other WSGI server, importing `wsgi.py` does it. Either way it happens exactly once per boot, before the first request.

The hourly database backup thread is started in the same hook, for the same reason: one schedule, owned by the process that outlives worker recycling.

2.6 Where configuration comes from

Everything parameterised lives in the `config/` package — that is coding rule **R1** — and each module reads its values from the environment with a shipped default:

Module	Owns
`config/database.py`	Host, port, database name, the owner and app roles, the protected-database list, the migration tokens and lock key.
`config/web.py`	Bind address, worker model, timeouts, logging.
`config/identity.py`	Which request headers carry the authenticated identity.
`config/permissions.py`	Who may author a change; the role vocabulary.
`config/theme.py`	Every colour, type size, spacing step, radius and shadow.
`config/reporting.py`	How a generated PDF is set, and its shipped wording.
`config/files.py`	Upload directory, allowed extensions, size limits.
`config/paths.py`	Repository-relative paths (migrations, contracts, static).
`config/audit.py`	Audit thresholds and scan scope.
`config/backup.py`	The backup schedule.
`config/governance/`	The governance vocabulary and field metadata.

The `.env` trap — read this before running anything ad hoc

`config/env.py` loads `.env` *at import time*, so *any* process that imports `config` inherits that file's settings — including `DB_NAME` pointing at the live database. A throwaway `python -c "from src.web import create_app"` is aimed at production by default. A real environment variable always beats the file, so the fix is to set the target explicitly and print what you actually resolved to before the first write:

```
python -c "from config.database import DatabaseConfig; \
          print(DatabaseConfig.from_env().name)"
```

This is coding rule **R9**, and it exists because two ad-hoc debugging scripts wrote junk models into the live, append-only governance log on 2026-08-06. Those records are permanent; removing them from the active inventory took a committee.

2.7 The console and the documents

The front end is deliberately dependency-free: vanilla JavaScript modules under `src/static/js/`, one stylesheet at `src/static/css/app.css`, and a single Jinja template `src/templates/index.html`. There is no build step, no bundler and no framework to keep current — a decision that matters for a system whose useful life is measured against a regulator's retention period rather than a framework's release cadence. Rule **R7** keeps those assets out of `src/web/`, and forbids embedding JavaScript or CSS inside a `.py` file at all.

PDFs — the MRC meeting pack, the Model Risk Report, the coding-rule audit reports, the governance completeness audit — are generated as LaTeX and compiled by **tectonic**, which must be on `PATH`. Without it those endpoints return `503` and everything else works. The LaTeX source of a generated document can be filed alongside its PDF as the auditable original; uploads are only ever stored and served back, never compiled, so no user-supplied LaTeX is executed on the server.

3 Changing the style

3.1 The rule this section is about

An adopting institution has to be able to put its own brand on the platform without touching the front end. That is coding rule **R10**, added on 2026-08-22, and it is absolute: **no colour literal — hex, `rgb()`, `rgba()`, `hsl()`, or the bare `RRGGBB` spelling LaTeX uses — may appear anywhere except `config/theme.py`**. Not in the CSS, not in a template, not in a `.js` file, not in a `.py` file, and not in the PDF preamble.

The rule exists because the alternative had already failed. Before it, the brand blue existed as `#1976d2` in the stylesheet and the JavaScript and as `#1565C0` in the PDF preamble, in two letter-cases: one decision, written down four times, already disagreeing with itself. A rebrand was a find-and-replace across three languages and it would have missed some.

3.2 Where the values live

`config/theme.py` — one file, around 280 lines, organised in groups:

Group	What it holds
`BRAND`	The adopter's identity: `accent` (links, active tabs, focus, selection), its soft/border/ink/wash variants, and the header gradient. *The group a rebrand almost always touches, and often the only one.*
`SURFACE`	Paper: background, panel, raised, sunken, control, read-only, code, and the two rule weights.
`TEXT`	Ink: primary, secondary, muted, inverse.
`RAG`	Red / Amber / Green. This is the *product's vocabulary*, not a colour choice — the ratings are named in `VALID_RAG_RATINGS` and a division may recolour its own bands through the risk policy. These are the platform fallbacks.
`STATE`	Banner and chip tints keyed to the state they announce (danger, warn, ok).
`HUE`, `GRAPH`	Hues that distinguish a state *without* asserting a risk rating — a Product being "Proposed" is a lifecycle fact, and drawing it in a RAG colour would read as a rating.
`TYPE`, `SPACE`, `RADIUS`, `SHADOW`	The type scale, the spacing steps, corner radii and shadows. Both typefaces are tokens too.

Two further tables in the same file are about *meaning* rather than value:

- `STATUS_COLOUR_TOKENS` — which *token* each status is drawn in, per domain (`rag`, `meeting_status`, `lifecycle`, `product_status`, `mrc_state`, `validation_question`, `mission_criticality`, `review_thread`). It names a token, never a colour. Six maps in `helpers.js` used to hold 27 entries between them; this is those six, unified.
- `STATUS_COLOUR_DEFAULTS` — what an *unrecognised* value is drawn in, per domain. A domain absent from it has no default on purpose: an unknown rating must render as a dash, not as a grey badge asserting a classification nobody made.

Keys are the CSS custom-property name without the leading `-`, so a token is spelled the same way in `config/theme.py`, in the stylesheet, and in an adopter's override. There is no translation layer to drift.

3.3 How a token reaches the screen

```
config/theme.py                 the values (THEME, a flat dict of token -> value)
      |
      +--> src/web/_theme.py --> GET /theme.css        :root { --accent: #1976d2; ... }
      |          |                    linked BEFORE app.css, cache-busted by a
      |          |                    SHA-256 of the served text (theme_version)
      |          |
      |          +------------> GET /api/theme         { tokens, status_colours,
      |                           hydrates the JS       status_defaults }
      |                           'Theme' object
      |
      +--> config/reporting.py --> src/governance/latex_style.py --> the PDFs
```

`/theme.css` is a *route*, not a checked-in file, for two reasons. A generated file would have to be regenerated and committed whenever a token changed, and the two would drift the first time somebody forgot. And a route sidesteps the Jinja template cache: with `debug=False` a new `<script>` or `<link>` tag in `index.html` is not picked up until the server restarts, but the CSS a route serves is fresh on every request.

Its cache-buster is a hash of the served text rather than the file-modification stamp used for static assets — `asset_version` watches files under `src/static`, and the theme is not one. Hashing gives the same guarantee: the URL changes exactly when the bytes do, so a rebrand cannot be masked by a browser cache, and an unchanged theme keeps its cached copy across restarts.

In JavaScript, three accessors on `Theme` (in `src/static/js/helpers.js`):

Call	Use it when
`Theme.colour(domain, value)`	Colouring a status — a rating, a meeting state, a lifecycle. The mapping is server-supplied.
`Theme.ref(token)`	You need a reference to a token: emits `var(-token)`.
`Theme.hex(token)`	Only where the value is *manipulated* rather than used — the heatmap appends two hex digits of alpha, and `var(-red)22` means nothing.

3.4 How a token reaches the PDFs

`config/reporting.py` derives the document colours from the theme:

```
DOCUMENT_LINK_COLOUR           = BRAND["accent"]   # hyperlinks, headings
DOCUMENT_CHROME_COLOUR         = TEXT["muted"]     # running header/footer furniture
DOCUMENT_CLASSIFICATION_COLOUR = RAG["red"]        # the CONFIDENTIAL banner
DOCUMENT_UNRATED_COLOUR        = RAG["grey"]       # a band with no colour of its own
```

`src/governance/latex_style.py` converts them for `xcolor`'s HTML model and applies them; `latex.py` builds structure only. So a report and the screen it came from cannot disagree. The same file also owns the page setup taken from the house template — geometry, line spacing, rule width, the title fonts — and `DOCUMENT_BODY_FONT`, which is empty by default (Computer Modern, the serif face every generated document has printed in to date) and can be set to `"sans"` to load Helvetica and match the console. That is a visible change to every document the platform produces, so it is a decision an adopter makes rather than a side effect.

The institution's *words* on those documents — the classification banner, the "prepared by" line, the legal notice, the organisation name — are settings, recorded in **Admin → Misc**, and re-label every document at once. The values in `config/reporting.py` are what a fresh install prints until it does.

3.5 Rebranding: the procedure

1. **Edit `config/theme.py`.** For most adopters this is the `BRAND` group and nothing else — `accent` plus its four variants and the three header values. Keep the token *names*; change the values.
2. **If the type or spacing scale changes,** edit `TYPE`, `SPACE`, `RADIUS`, `SHADOW` in the same file. Values are strings, deliberately — a token is whatever CSS accepts in that position, so `"13px"`, `"0 1px 3px rgba(0,0,0,.12)"` and a whole font stack are all legal.
3. **If a status should be drawn in a different token,** edit `STATUS_COLOUR_TOKENS` — not a map in a `.js` file.
4. **Upload the organisation's logo** through the console (Admin); the logo is user-uploaded, not a file in the repository, and it flows into the generated title pages.
5. **Set the document wording** in Admin → Misc: organisation name, classification banner, author line, legal notice.
6. **Run the styling audit.** `python -m audit` and read the final line. Three gated checks sit at zero: a colour literal in an asset, a colour literal in Python, and a `var(-token)` naming a token the theme does not define.
7. **Restart the server and look at it.** The theme route is not cached, but `index.html` is under `debug=False`, so restart — or `kill -HUP` the gunicorn master to recycle its workers — if you touched a template. Verify on the scratch database at port 5016 (`scripts/restore_scratch.sh`, then `scripts/dev_app.sh`), never against the live estate.
8. **Regenerate one PDF** and check the brand followed. The Audit Reports tab generates them in-app; `python -m audit -pdf` does it from the command line.

Three traps, all of which have caught somebody

An undefined token fails silently. A `var(-token)` the theme does not define is not an error — the browser drops the declaration and the element renders with whatever it inherits. A rename therefore reaches a screen as a colour quietly falling back on whichever control nobody happened to open. This is why the audit checks token references as well as literals, and why you rename by adding the new token and migrating to it rather than renaming in place.

SVG presentation attributes do not accept `var()`. `fill="var(-accent)"` renders black. Anything drawing into a canvas uses a CSS class or a `style` attribute.

Many UI surfaces are hand-curated, not spec-driven. Adding a *field* to config is not enough — it has to be wired into the relevant detail lists, the Admin tab bar and the register form. Colours are the exception, not the pattern: they genuinely do flow from one file.

3.6 What is not built yet

`docs/design/theme_config.md` §4.3 proposes two ways for an adopter to override the theme without editing code. Neither is built at the commit on the title page:

- **A theme file** — `THEME` shipped as defaults, overridden by JSON at `MODELRISK_THEME=themes/<adopter>.json`. Designed; the environment variable is not read anywhere in the code today.
- **Admin → Branding** — stored like the MRC settings. Deferred deliberately.

So a rebrand today is a one-file source change, committed and deployed like any other — which for a vendor cloning the platform into their own suite is usually acceptable, and is in any case a change the audit will hold honest. If you need the JSON override, it is a small piece of work: read the file in `config/theme.py` after the groups are assembled, merge over `THEME`, and leave everything downstream untouched.

4 The coding rules R1–R10

4.1 Why an integrator should care

The rules are not house style. Each of the four jobs in this guide is cheap or expensive depending on whether a particular rule held:

- Rebranding is a one-file change *because* **R10** holds.
- Pointing the platform at a different database is configuration *because* **R1** holds; porting it to a different *engine* is bounded *because* **R6** holds.
- Containerising is straightforward *because* **R1** puts every parameter behind an environment variable.
- Putting your own SSO in front of it is a header-name change *because* **R1** holds for identity too.

The full text is `docs/rules/coding_rules.md`; each rule there carries the rule, why it exists, how to comply, and how it is verified. This section is the operator's summary.

4.2 The rules

	Rule	Enforcement
R1	No hardcoded parameters outside `config/`. Filenames, ports, header names, field lists, value sets, thresholds, default modes. Derive, don't duplicate.	**Gated** by the hardcoding scan.
R2	No non-test source file over **300** lines; test files over **400**. Split into a package of focused modules.	**Gated** for `.py` repo-wide (`config`, `src`, `scripts`, `app`, `docs` plus root files) and for tests. JS/CSS are reported, not gated.
R3	≥**99%** line+branch coverage, checked every stage.	**Gated** by `fail_under = 99` in `pyproject.toml`; the audit prints the current figure and the weakest packages.
R4	No functions in `__init__.py` — imports and `__all__` only.	**Gated**. The audit lists every package initialiser with its length and its re-export verdict.
R5	Every first-party file carries the canonical licence header or a one-line notice; files that cannot hold a comment get a sibling `.license`.	**Deliberately not automated.** Whose name belongs at the top of a file is a commercial question, and a failing build forces it at the wrong moment. The maintainer runs a local-only tool.
R6	Never touch the data store directly. All persistence through `src/database/`; no SQL, cursor, connection or driver import anywhere else. No escape hatch.	**Reported**, in three scans: data access outside the package, path definitions outside `config`, and `.json` file I/O under the source tree. Treated as zero-tolerance in review.
R7	Front-end assets under `src/static` and `src/templates`; never JS or CSS embedded in a `.py` file.	**Gated** for embedded JS/CSS; the directory layout is reviewed, and a misplaced asset shows up as a 404 in the e2e smoke suite.
R8	No duplication — extract a named utility. Zero tolerance across two models.	**Reported**: jscpd's duplication percentage (threshold 5%, clones of ≥8 lines / 50 tokens). Cross-model duplication is a hard fault in review.

	Rule	Enforcement
R9	Test and verify against a restored backup, never the live database.	**Partially enforced.** `assert_disposable` guards the destructive test fixtures; `scripts/dev_app.sh` refuses to start against a protected database. An ordinary write to a protected database is not stopped — that gap is what the known-actor rule (§7) narrows.
R10	Every visual parameter comes from `config/theme`.	**Gated** on three checks, all at zero: a colour literal in an asset, a colour literal in Python, and a `var(-token)` naming an undefined token.

4.3 Running the audit

```
python -m audit           # console report; exits non-zero on a gated finding
python -m audit --pdf     # also renders one PDF per rule plus a consolidated overview
```

The last line is the verdict:

```
==================================================
PASS: 0 gated finding(s)
```

Read the line, not the exit code

A background run that reports "completed, exit code 0" with empty output is **not** a pass — that exit code belongs to the shell that launched it. Always read the final `PASS:` / `FINDINGS:` line.

Sections marked `OK` have no findings and `!!` have some; the reported (non-gating) sections are printed after the gated ones and are not counted in the total. Coverage is read from `coverage.xml` if one is present, so for a figure rather than a reminder, run `coverage run -m pytest && coverage xml` first. Duplication needs Node and `jscpd`; without them that line says so and the audit continues.

4.4 The reports

`-pdf` — and the console's **Audit Reports** tab, which calls the same function so both emit exactly the same set — writes one PDF per rule plus a consolidated overview:

```
audit_r1_hardcoded_parameters.pdf     audit_r7_embedded_js.pdf
audit_r2_large_files.pdf              audit_r8_duplicate_code.pdf
audit_r3_test_coverage.pdf            audit_r10_styling.pdf
audit_r4_init_files.pdf               json_files_report.pdf
audit_r6_data_access.pdf              coding_rule_audit.pdf    (the overview)
audit_r6_path_definitions.pdf
```

They are written to `AUDIT_OUTPUT_DIR`, which is `$UPLOAD_DIR/audit/<database name>` — under the configured upload directory, so generated state follows `UPLOAD_DIR` wherever an operator points it rather than being written into the checkout, and scoped by database so reports from a demo and a live instance never masquerade as one another. Rendering needs tectonic; without it the audit says so and returns non-zero for that reason alone.

Alongside them the audit run drops its test evidence — `junit.xml`, `coverage.xml`, the browsable HTML coverage report, and `init_audit_results.json` — so one directory holds the whole quality story for a given database.

4.5 Where the thresholds live

Every number in this section is a value in `config/audit.py`, not a literal in the audit code (**R1** applies to the audit too): `MAX_SOURCE_FILE_LINES = 300`, `MAX_TEST_FILE_LINES = 400`, `COVERAGE_MINIMUM = 99`, `DUPLICATION_MAX_PERCENT = 5.0`, `JSCPD_MIN_LINES = 8`, `JSCPD_MIN_TOKENS = 50`. The scan scope is there too: `SOURCE_DIRS`, `REPO_SCAN_DIRS`, `STYLING_ASSET_DIRS`, `STYLING_GATED_SUFFIXES`, `EXCLUDE_DIRS`.

An adopting institution that wants a different bar changes those values. Be aware that loosening one is visible in the generated report — the PDFs print the threshold they were run against, which is the point.

4.6 The baseline

A rule set is only useful if "clean" means something. At the commit on the title page the gate sits at **2 findings**, both R2, both known:

```
384  src/governance/client/pack.py
326  src/web/api_mrc/lifecycle.py
```

Anything beyond those two is new work's doing. Diff against `main` before assuming otherwise — the list moves as files are split.

5 Switching to a different SQL server

5.1 Two different questions

"Switching the SQL server" means one of two things, and they are days apart in effort.

A different PostgreSQL server — configuration only. A managed instance (RDS, Cloud SQL, Azure Database for PostgreSQL), a different host, port, database name or role set is **no code change at all**. Seven environment variables, read in `config/database.py`:

```
DB_HOST=db.internal.example.com
DB_PORT=5432
DB_NAME=modelrisk
DB_OWNER_USER=modelrisk_owner        # owns the schema, runs migrations
DB_OWNER_PASSWORD=...
DB_APP_USER=modelrisk_app            # the runtime role; append/read only on events
DB_APP_PASSWORD=...
# DB_MAINTENANCE_NAME=postgres       # only used while creating a database
```

Start the app; `bootstrap()` applies the migrations, seeds the admin reference lists on a fresh database, and rebuilds the read models. Two constraints on the names you choose: `DB_APP_USER` must be a plain SQL identifier (the migrations grant to it by name and the runner refuses anything else), and `DB_NAME` must not collide with a name in `PROTECTED_DB_NAMES` unless it really is the estate — that list is what the test fixtures and the verification app refuse to touch.

A different engine — a port. Microsoft SQL Server, Oracle, MySQL. That is real work, and the rest of this section is about scoping it honestly. **Nothing in the repository implements a second engine today.** What exists is `docs/mssql-immutability.md`, which covers the one part of a SQL Server port that is easy to get wrong and impossible to notice.

5.2 What is engine-specific, and where

The good news is that rule **R6** makes the answer enumerable. Every module outside `src/database/` reaches the store through named functions, so nothing above that line knows what engine it is talking to. The engine-specific surface is:

Where	What is PostgreSQL-specific
`src/database/_connection.py`	The driver. `psycopg` imports, the `connect()` calls, the autocommit admin connection, and `CREATE DATABASE` via `psycopg.sql.Identifier`. This is the only module that opens a connection.
`src/database/eventstore.py`	The SQL itself. `%s` parameter markers, `INSERT ... RETURNING`, and `psycopg.types.json.Jsonb` for the payload column.
`src/database/_migrate.py`	`pg_advisory_xact_lock` for the one-migrator-at-a-time guarantee, and `TIMESTAMPTZ`/`now()` in the `schema_migrations` DDL.
`src/database/migrations/*.sql`	Eleven files of PostgreSQL DDL: `BIGINT GENERATED ALWAYS AS IDENTITY`, `UUID` with `gen_random_uuid()`, `JSONB`, `TIMESTAMPTZ`, the `DO $$ ... $$` block that creates the role, and `GRANT`/`REVOKE` semantics.
`src/database/projections.py`	Read-model upserts (`ON CONFLICT`) and any JSON operators used to query the payload.

Where	What is PostgreSQL-specific
`scripts/pg.sh`, `scripts/backup_db.sh`, `restore_scratch.sh`	`pg_ctl`, `pg_dump`, `psql`. Operational, not application code, but a deployment without a working backup is not a deployment.

Everything else — roughly the whole application — is untouched. That is the dividend R6 was written to pay.

5.3 A port, step by step

1. **Do not add T-SQL to `src/database/migrations/`.** The runner globs `*.sql` and applies *every* file in lexical order as PostgreSQL; a T-SQL file dropped in there is executed against Postgres and fails the migration for every existing deployment. Give the new engine its own migrations directory and select it in `config/paths.py`.
2. **Introduce a backend seam inside `src/database/`.** Keep the public names — `append_event`, `read_stream`, `read_view`, `replay`, `run_migrations` — and put the dialect behind them. A `DB_BACKEND` variable in `config/database.py` selects it (**R1**: the choice is a parameter, not an import somebody edits).
3. **Translate the schema.** `IDENTITY` for `global_seq`, `UNIQUEIDENTIFIER`/`NEWID()` or an application-generated UUID for `event_id`, `NVARCHAR(MAX)` with an `ISJSON` check for the payload, `DATETIMEOFFSET` for the timestamps. Keep the unique constraint on `(aggregate_type, aggregate_id, stream_seq)`: it is the concurrency backstop, not an index.
4. **Replace the advisory lock.** On SQL Server, `sp_getapplock` with `@LockOwner = 'Transaction'` is the direct equivalent. Do not skip it because "we only run one process" — gunicorn forks, and this is exactly the failure that looks like a worker crashing on boot for no reason.
5. **Re-establish immutability at the grant level.** This is the step that is easy to lose, and it has its own document. See below.
6. **Fix the backups.** `scripts/backup_db.sh` locates `pg_dump` and **exits 0 when it cannot find one**, so on another engine the hourly schedule reports success while doing nothing. Replace it, and verify a restore, not just a dump.
7. **Run the suite against the new engine.** `pytest` derives its own disposable database and rebuilds every projection, so a passing suite is a genuine statement about the port. Then `python -m audit` — a port that leaks a driver import outside `src/database/` shows up in the R6 section.

5.4 Immutability does not cross the border

The platform's central claim is that the event log is the system of record and cannot be rewritten. On PostgreSQL that is enforced by the database:

```
GRANT SELECT, INSERT ON events TO {{app_user}};
REVOKE UPDATE, DELETE, TRUNCATE ON events FROM {{app_user}};
```

A driver swap moves the queries. It does not move the grants — those are DDL that the PostgreSQL migration applies. Until the equivalent is in place on the new engine, the runtime account can rewrite the log: the hash chain will still *detect* it, but nothing *prevents* it, and "enforced at the database-grant level" is a claim you may be making to an assessor.

On SQL Server there is one difference that turns a correct-looking script into a decorative one. **`REVOKE` only removes a permission granted directly**; it does not override a permission inherited through role membership. If the application user is a member of `db_datawriter` — a very common way to set up an app account — `REVOKE UPDATE ON events` changes nothing at all and the log

stays writable. `DENY` is the one that wins. Two smaller differences: there is no `TRUNCATE` permission (it is authorised by `ALTER`, so you `DENY ALTER`), and `sysadmin` bypasses permission checks entirely — as will `db_owner` — so check what the app connects as before trusting any of it.

```
GRANT SELECT, INSERT ON dbo.events TO [mkm_app];
DENY UPDATE, DELETE ON dbo.events TO [mkm_app];
DENY ALTER ON dbo.events TO [mkm_app];      -- TRUNCATE is authorised by ALTER
```

The projections stay mutable — they are derived and rebuildable, exactly as on PostgreSQL. `docs/mssql-immutability.md` carries the full script, the two privilege checks that prove the account is not privileged enough to ignore the `DENY`, and the functional test that an assessor should actually accept:

```
EXECUTE AS LOGIN = N'mkm_app';
BEGIN TRY
    UPDATE dbo.events SET actor = actor WHERE 1 = 0;
    PRINT '*** FAILED: the event log is writable ***';
END TRY
BEGIN CATCH
    PRINT 'Blocked as expected: ' + ERROR_MESSAGE();
END CATCH
REVERT;
```

`WHERE 1 = 0` matches no rows — SQL Server checks the permission on the object regardless, so the test cannot alter data even if the `DENY` were missing. Expect error 229, *the UPDATE permission was denied on the object 'events'.*

Two things immutability grants do not cover

The **owner / migration account can still alter the log**, by design — it has to be able to create the schema. Immutability is a property of the *runtime* account, which is why the two roles are separate configuration values and why the owner credentials should not be in the application's environment beyond the moment migrations run.

The **hash chain is the detection half** and is application-level, so it crosses to any engine unchanged. The grants are the prevention half. An audit story wants both.

6 Running it inside a container

6.1 Status: not shipped

There is **no `Dockerfile` or Compose file in the repository** at the commit on the title page. The platform is developed and deployed natively — the README says so explicitly ("PostgreSQL 16, run natively, no Docker"). The listings below are a worked starting point that follows the application's actual startup contract; they are not something the test suite exercises, so treat them as a design you own rather than as a supported artefact.

The application itself is container-friendly for one reason: **R1**. Every parameter is already an environment variable with a shipped default, so there is nothing to bake into an image and no file to template.

6.2 What has to be in the image

Component	Required?	Why
Python 3.13+	Yes	`requires-python = ">=3.13"`.
`psycopg[binary]`, `flask`, `gunicorn`, `jsonschema`	Yes	The runtime dependencies. `gunicorn` needs `fcntl`, so it is POSIX-only — on Windows the development server is the only option.
tectonic	Effectively	The LaTeX engine. Without it every PDF endpoint — meeting pack, Model Risk Report, audit reports — returns `503`. For a governance platform whose output is committee papers, that is not really optional.
pandoc	Optional	Only the User Guide PDF in the Audit Reports tab.
`postgresql-client`	Conditional	`scripts/backup_db.sh` needs `pg_dump`. Omit it *and* set `BACKUP_ENABLED=0` — see below.
Node + `jscpd`	No	Only the duplicate-code section of `python -m audit`; keep it out of a runtime image.

6.3 A worked Dockerfile

```
FROM python:3.13-slim AS base

# tectonic renders every PDF the platform produces; pandoc is only the user guide.
# postgresql-client provides pg_dump for the in-app backup schedule -- drop both it
# and BACKUP_ENABLED if the database is backed up by the platform hosting it.
RUN apt-get update && apt-get install -y --no-install-recommends \
        tectonic pandoc postgresql-client ca-certificates \
    && rm -rf /var/lib/apt/lists/*

WORKDIR /app

COPY requirements.txt ./
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

# Warm tectonic's package cache at BUILD time, so the first PDF request in
# production does not have to reach the internet. Without this a locked-down
# network turns the first report into a timeout rather than a 503.
ENV TECTONIC_CACHE_DIR=/opt/tectonic-cache
RUN mkdir -p "$TECTONIC_CACHE_DIR" \
    && printf '%s\n' \
```

```
        '\documentclass{article}' \
        '\usepackage{xcolor,longtable,booktabs,geometry,fancyhdr,hyperref}' \
        '\begin{document}warm\end{document}' > /tmp/warm.tex \
    && tectonic -o /tmp /tmp/warm.tex && rm -f /tmp/warm.*

# Run as a non-root user that owns the state directory.
RUN useradd --system --uid 10001 --home /app modelrisk \
    && mkdir -p /data/uploads /opt/tectonic-cache \
    && chown -R modelrisk:modelrisk /data /opt/tectonic-cache
USER modelrisk

# The container's network namespace is not the host's: bind the wildcard address,
# not the 127.0.0.1 that config/web.py defaults to.
ENV WEB_HOST=0.0.0.0 \
    WEB_PORT=5015 \
    UPLOAD_DIR=/data/uploads \
    PYTHONUNBUFFERED=1

EXPOSE 5015
VOLUME ["/data/uploads"]

# /api/models proves two things at once: the process serves, and it can reach
# the database. There is no dedicated /health route. The start period covers
# migrations and the read-model replay, which run before the first request.
HEALTHCHECK --interval=15s --timeout=10s --start-period=60s --retries=5 \
  CMD python -c "import urllib.request as u; \
      u.urlopen('http://127.0.0.1:5015/api/models')" || exit 1

# gunicorn.conf.py runs bootstrap() in the master before the fork: migrations,
# seeded reference lists, read-model rebuild. Workers inherit the done-marker.
CMD ["gunicorn", "--config", "gunicorn.conf.py", "wsgi:application"]
```

6.4 A worked Compose file

```
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_USER: mkm_owner            # must equal DB_OWNER_USER below
      POSTGRES_PASSWORD: ${DB_OWNER_PASSWORD:?set it}
      POSTGRES_DB: mkm_modelrisk
    # The app opens workers x threads connections; the stock max_connections is 100.
    command: ["postgres", "-c", "max_connections=200"]
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U mkm_owner -d mkm_modelrisk"]
      interval: 5s
      timeout: 3s
      retries: 20

  app:
    build: .
    depends_on:
      db:
        condition: service_healthy       # migrations run at boot; the DB must be up
    environment:
      DB_HOST: db
      DB_PORT: 5432
```

```
      DB_NAME: mkm_modelrisk
      DB_OWNER_USER: mkm_owner
      DB_OWNER_PASSWORD: ${DB_OWNER_PASSWORD:?set it}
      DB_APP_USER: mkm_app              # created by migration 0002 with these grants
      DB_APP_PASSWORD: ${DB_APP_PASSWORD:?set it}
      IDENTITY_HEADER: X-Auth-Request-User
      ENFORCE_KNOWN_ACTORS: always      # see section 7
      IDENTITY_DEV_FALLBACK: unauthenticated   # deliberately not a registered person
      BACKUP_ENABLED: 0                 # back the volume/database up outside the app
      WEB_WORKERS: 4
      WEB_THREADS: 4
      WEB_TIMEOUT: 300                  # tectonic renders take far longer than 30s
    volumes:
      - uploads:/data/uploads
    ports:
      - "127.0.0.1:5015:5015"           # never publish this straight to a network
    # The health check is declared in the Dockerfile (HEALTHCHECK) and inherited.

volumes:
  pgdata:
  uploads:
```

6.5 The things that only bite once it is in a container

Bind the wildcard address. `config/web.py` defaults `WEB_HOST` to `127.0.0.1`, which inside a container means "reachable by nothing". Set `WEB_HOST=0.0.0.0` (or `WEB_BIND` directly) and put the access control at the proxy, not at the bind address.

Boot is not instant, and it touches the database. The gunicorn master applies migrations, seeds reference lists on a fresh database, and *replays every projection* before it forks. On a large estate that is the slowest thing the process ever does. Give the health check a `start_period`, make the app wait for the database rather than restart-looping, and if replay-per-boot becomes unacceptable set `REBUILD_VIEWS_ON_STARTUP=0` — knowing that you have then taken responsibility for rebuilding a view after a restore.

One master, many workers — do not scale the container blindly. Two things are deliberately once-per-boot in the master: the bootstrap and the backup schedule. Running three replicas gives you three backup schedules and three processes racing the same migrations. The advisory lock makes the race safe, not free. If you need horizontal scale, run one instance with the schedule enabled and the rest with `BACKUP_ENABLED=0`, or lift bootstrap into an init container / `Job` and start the app with migrations already applied.

Size the pool against `max_connections`. Peak connections are `WEB_WORKERS` × `WEB_THREADS` — 16 by default — opened and closed per call with no shared pool. The defaults are deliberately modest rather than the usual $(2 \times \text{cores}) + 1$, because that formula on a many-core host exhausts a stock `max_connections` of 100.

PDFs need time and a warm cache. `WEB_TIMEOUT` is 300 seconds for exactly one reason: rendering shells out to tectonic, and under gunicorn's stock 30-second timeout the worker is killed mid-render and the request fails as a bare `502` with nothing in the log to explain it. Tectonic also fetches LaTeX packages on first use — warm the cache at build time (above) or the first report in a network-restricted environment hangs rather than failing cleanly.

Uploads are state; the container is not. `UPLOAD_DIR` holds every uploaded document and every generated report (audit PDFs land in `$UPLOAD_DIR/audit/<database name>`). It must be a volume, and it must be in your backup set — the event log records that a document exists and its metadata, not its bytes.

Do not bake `.env` into the image. `config/env.py` loads `.env` at import if it is present. A real environment variable always wins, so an `.env` that got copied in during `COPY . .` will not override your orchestrator's settings — but it *will* fill in anything you forgot to set, silently, including a `DB_NAME` pointing at somebody else's estate. Add it to `.dockerignore` and keep secrets in the platform's secret store.

Backups inside a container are a decision, not a default. `scripts/backup_db.sh` sources `.env` for its own credentials and shells out to `pg_dump`; if it cannot find one it exits 0, so a misconfigured image reports a healthy hourly schedule while writing nothing. In a container, prefer the database platform's own snapshots and set `BACKUP_ENABLED=0` — or keep the schedule, install `postgresql-client`, mount a backup volume, and *verify a restore*.

Verify against a scratch copy, not the estate. Rule **R9** does not relax because the app is in a container. A verification instance points `DB_NAME` at a scratch database restored from a backup; the guard in `scripts/dev_app.sh` that refuses to start against a protected name is a shell script, so a container that invokes gunicorn directly does not inherit it. Either keep that entry point or re-implement the check in yours.

7 Permissions and SSO

7.1 The model in one sentence

The platform does not authenticate anybody. Identity is established at the perimeter — by your SSO, a reverse proxy, an identity-aware gateway — and forwarded to the application in a request header, which the application *trusts verbatim* and records as the author of every event it writes.

That is a legitimate architecture, and it is the same one oauth2-proxy, Google IAP, Azure Application Gateway and most Kubernetes ingress-auth setups assume. It has exactly one requirement, and §7.4 is about taking it seriously: **if anything can reach the application port without passing through the proxy, it can author governance history under any name.**

7.2 What arrives, and what happens to it

```
config/identity.py
  IDENTITY_HEADER       = "X-Auth-Request-User"                  (the stable identifier)
  IDENTITY_NAME_HEADER  = "X-Auth-Request-Preferred-Username"    (display name, optional)
  IDENTITY_DEV_FALLBACK = "<an address>"   used when NO header is present
  SYSTEM_ACTOR          = "system"         the platform's own actor, for first-run seeding
```

`resolve_principal()` (`src/identity/resolver.py`) does a case-insensitive header lookup and returns a `Principal` carrying `actor`, `display_name` and a `source` — `sso_header`, `fallback` or `ingest`. It is a deliberate seam: the rest of the application needs only a `Principal`, so a different identity source can be substituted here without touching a single command handler.

Machine producers get a synthetic principal, `ingest:<source_repo>`, so an ingested payload can never be mistaken in the audit trail for a person's work.

7.3 Who is allowed to author a change

Phase P1 of `docs/permissions_design.md` is **built and enforced**: a change must be authored by somebody the institution has registered under **Admin → People**. The check lives in `authorize()` in `src/command/dispatch.py` — so every mutating command passes it, and a new endpoint cannot forget to ask — and in `request_actor()` for the upload paths that call their command directly. A refusal is an HTTP 403 naming the actor and the remedy.

Matching is on **name or email**, case- and whitespace-insensitive, because the console sends a person's name and an SSO deployment sends their address; either identifies them.

Policy is `config/permissions.py`:

Setting	Meaning
`ENFORCE_KNOWN_ACTORS`	
`protected` (default)	The rule applies only against a protected database — the real estate. Test and scratch databases stay open so the suite and throwaway environments are unaffected.
`always`	Every database. **What a hardened deployment sets.**
`never`	Disabled. A break-glass switch for an adopter mid-migration; it leaves the governance record open to any identity, so it should be temporary.
`ENFORCE_REVIEWER_INDEPENDENCE`	
`always` (default)	Segregation of duties: a model's peer reviewer may not be its own owner. Existing breaches are reported by the completeness audit either way; enforcement only refuses the write that would create or keep one, so no record is ever locked shut.

Setting		Meaning
`MACHINE_ACTOR_PREFIX ingest:`	=	Machine producers pass the known-actor rule without being people.
`ALLOW_WHEN_NO_ PEOPLE_REGISTERED`		While the People list is empty the rule stands down, so a fresh install can register its first person. **The first person added closes it.**
`ROLES`		The role vocabulary (owner, validator, committee, chair, administrator, reader), with the explanation shown beside each choice in Admin. An adopting institution re-sets this to its own framework.

Renaming a role un-assigns it

`ROLES` keys are stored on the People records. Changing a key's *name* without migrating the records that hold it silently un-assigns the role — exactly as renaming a risk band would. On a live estate, add the new role and migrate; never rename in place.

7.4 The trust boundary — the part to get right

The header is taken at face value. On 2026-08-06 the identity hard-coded in the web tests, `alice@corp`, authored 24 real events on the live estate from an ad-hoc script, and the audit trail recorded them faithfully because nothing was wrong with them — they were correctly attributed to the identity that was sent. P1 narrows *who* can author (a registered person) but does not verify that the caller *is* that person: anyone who can address the app can still send a registered name.

So the deployment has to close it. Five things, all outside the application:

1. **The app listens only where the proxy can reach it.** Loopback, a private network, or a Kubernetes `ClusterIP` — never a published port on a routable interface. The Compose example above binds `127.0.0.1:5015` for this reason.
2. **The proxy strips the identity headers from every inbound request** before setting its own. This is the single most important line of configuration in an SSO integration, and it is the one most often left out: without it a client simply sends `X-Auth-Request-User` itself.
3. **Set `ENFORCE_KNOWN_ACTORS=always`** rather than relying on the database name being one the platform happens to recognise as protected.
4. **Point `IDENTITY_DEV_FALLBACK` at something that is not a person.** When no header arrives the web layer falls back to this actor — it does not refuse. Set it to a value nobody will ever register (`unauthenticated`), and the known-actor rule turns every header-less write into a `403` instead of an event attributed to whoever the default names.
5. **Prefer a stable, non-reassignable identifier** in the header. An email address that can be reassigned to a new joiner makes historical attribution ambiguous years later, which for an append-only governance record is a real problem rather than a theoretical one.

7.5 Wiring it to an identity provider

The application only needs the header names to match, so any of these work. Set `IDENTITY_HEADER` and `IDENTITY_NAME_HEADER` to whatever your layer emits; the defaults follow the oauth2-proxy convention.

oauth2-proxy in front of nginx (OIDC — Entra ID, Okta, Ping, Keycloak).

```
server {
    listen 443 ssl;
```

```
    # 1. Never let a client supply its own identity.
    proxy_set_header X-Auth-Request-User                "";
    proxy_set_header X-Auth-Request-Preferred-Username  "";
    proxy_set_header X-Auth-Request-Email               "";

    location /oauth2/ {
        proxy_pass http://oauth2-proxy:4180;
        proxy_set_header X-Auth-Request-Redirect $request_uri;
    }

    location = /oauth2/auth {
        proxy_pass http://oauth2-proxy:4180;
        proxy_pass_request_body off;
        proxy_set_header Content-Length "";
    }

    location / {
        auth_request /oauth2/auth;
        error_page 401 = /oauth2/start?rd=$request_uri;

        # 2. Set them from what the auth subrequest returned.
        auth_request_set $user  $upstream_http_x_auth_request_user;
        auth_request_set $pref  $upstream_http_x_auth_request_preferred_username;
        proxy_set_header X-Auth-Request-User               $user;
        proxy_set_header X-Auth-Request-Preferred-Username $pref;

        proxy_pass http://modelrisk:5015;
        proxy_read_timeout 300s;    # match WEB_TIMEOUT: PDF renders are slow
    }
}
```

Other layers. Google IAP forwards `X-Goog-Authenticated-User-Email`; Azure App Service Easy Auth forwards `X-MS-CLIENT-PRINCIPAL-NAME`; an Istio/Envoy `ext_authz` filter emits whatever you configure. In each case point `IDENTITY_HEADER` at it and keep the strip-then-set discipline. Where the provider emits a signed assertion (a JWT) rather than a plain header, either have the gateway verify it and emit the plain header, or replace `resolve_principal()` — that is what the seam is for, and it is the only function that would change.

Then register the people. The known-actor rule matches against **Admin → People**, so the identities your IdP emits have to appear there, as each person's name or email. Two practical notes: the very first person can be added while the list is still empty (the rule stands down until then), and if you provision People in bulk from a directory export, use the same string form the header will carry.

7.6 What is designed but not built

Phase	What it is	Status
P1	Know who is real — a change must be authored by a registered person.	**Built**, 2026-08-07.
P2	Roles as data on the People record; assignment is itself an event.	Vocabulary and `roles_for()` exist; nothing consumes them but the Admin screen that sets them.

Phase	What it is	Status
P3	`authorize()` implemented in *report-only* mode — log refusals for a week of real use.	Designed. This is the phase that stops a permissions rollout blocking the day job.
P4	Enforce: refusals become `403` naming the missing role.	Designed.
P5	Segregation of duties — an owner cannot approve their own model; a validator cannot validate one they own; an administrator cannot ratify the policy they drafted.	Reviewer independence is enforced; the rest is designed.
P6	Machine actors scoped to append-only.	Designed.

The design's organising principle is worth stating, because it constrains how you should extend it: **permissions come from the domain, not a parallel user store**. Who is a real person is the People list; who owns a model is the model's owner field; who sits on the committee that governs it is the MRC's membership. There is no user table and no second role store to keep in step — which also means permissions are themselves auditable and ratifiable, exactly like everything else in the record.

When you do implement P3–P5, the permission map belongs in `config/permissions.py` as data (**R1**) — `COMMAND_ROLES = {"update_field": ("owner", "administrator"), "approve_signoff": ("committee",), ...}` — and the enforcement belongs in `authorize()`, which every mutating command already passes through. The UI should stop offering what the caller may not do, but that is cosmetic: the server is the control.

A Environment variable reference

Every value below has a shipped default; nothing must be set for the platform to start on a developer's machine. Real environment variables always beat `.env`.

Variable	Default	Notes
Database — `config/database.py`		
`DB_HOST`	`localhost`	
`DB_PORT`	`5433`	The repo's local cluster; a stock server is 5432.
`DB_NAME`	`mkm_modelrisk`	*Protected*: destructive fixtures and the verification app refuse it.
`DB_OWNER_USER` / `_PASSWORD`	`mkm_owner`	Owns the schema, runs migrations. Keep out of the runtime environment where you can.
`DB_APP_USER` / `_PASSWORD`	`mkm_app`	The runtime role. Must be a plain SQL identifier — the migrations grant to it by name.
`DB_MAINTENANCE_NAME`	`postgres`	Connected to only while creating another database.
`TEST_DB_BASE`	derived	A dedicated `*_test` base name for the suite.
`SCRATCH_DB`	`mkm_modelrisk_scratch`	Read by `scripts/dev_app.sh`.
Web — `config/web.py`		
`WEB_HOST`	`127.0.0.1`	**Set to `0.0.0.0` in a container.**
`WEB_PORT`	`5015`	5016 is the verification app by convention.
`WEB_BIND`	`$WEB_HOST:$WEB_PORT`	As gunicorn spells it.
`WEB_WORKERS` / `WEB_THREADS`	`4` / `4`	Peak DB connections are their product.
`WEB_WORKER_CLASS`	`gthread`	Requests wait on PostgreSQL; threads absorb that cheaply.
`WEB_TIMEOUT`	`300`	Generous on purpose — tectonic renders.
`WEB_GRACEFUL_TIMEOUT`	`30`	
`WEB_ACCESS_LOG` / `WEB_ERROR_LOG`	`-`	The standard streams.
`WEB_LOG_LEVEL`	`info`	
`MODELRISK_BOOTSTRAP_DONE`	unset	Set by the master after bootstrap; workers inherit it. Do not set it yourself.
`REBUILD_VIEWS_ON_STARTUP`	`1`	`0` skips the projection replay at boot.
Identity and permissions — `config/identity.py`, `config/permissions.py`		
`IDENTITY_HEADER`	`X-Auth-Request-User`	
`IDENTITY_NAME_HEADER`	`X-Auth-Request-Preferred-Username`	
`IDENTITY_DEV_FALLBACK`	a developer address	Used when no header arrives. Point it at a non-person in production.

Variable	Default	Notes
SYSTEM_ACTOR	system	The platform's own actor for first-run seeding.
ENFORCE_KNOWN_ACTORS	protected	always for a hardened deployment; never is break-glass.
ENFORCE_REVIEWER_INDEPENDENCE	always	never for an adopter mid-migration.
State and backups — config/files.py, config/paths.py, config/backup.py		
UPLOAD_DIR	<repo>/uploads	Documents *and* generated reports. Must be a volume; must be backed up.
PG_DATA_DIR	./.pgdata	Only the repo-local cluster.
PRODUCER_ROOT	sibling clone	Root of a producing repository for ingest migrations.
BACKUP_ENABLED	1	0 for anything that is not the real deployment.
BACKUP_INTERVAL_SECONDS	3600	
BACKUP_STARTUP_DELAY_SECONDS	20	So migrations and the rebuild have settled first.
BACKUP_TIMEOUT_SECONDS	300	An unfinished dump is abandoned; the next tick retries.

B Verification checklist

Run these before calling an integration done. Each one answers a question somebody will ask later.

Command	Answers
python -c "from config.database import DatabaseConfig; print(DatabaseConfig.from_env().name)"	Which database this process actually resolved to. Run it before any ad-hoc write.
pytest	Unit + integration, on a database the suite derives for itself.
coverage run -m pytest && coverage report	The ≥99% gate (R3).
python -m audit	The coding-rule gate. Read the final PASS: / FINDINGS: line.
python -m audit -pdf	That tectonic works and the reports render.
pytest tests/e2e -m e2e	The browser suite, against a live server on 5016. Run one at a time — it truncates a shared database.
curl -i http://host/api/models	That the app serves and reaches the database.
curl -i -H 'X-Auth-Request-User: nobody@example.com' -X POST http://host/api/...	That an unregistered actor is refused with 403 — i.e. the known-actor rule is in force.
curl -i -H 'X-Auth-Request-User: someone' http://proxy/...	That your proxy *strips* a client-supplied identity header rather than forwarding it. The single most important check in §7.

C Where things live

config/	every parameter (R1). database, web, identity, permissions, theme, reporting, files, paths, audit, backup, governance/
src/database/	the ONLY module that talks to the store (R6). eventstore.py, projections.py, _connection.py, _migrate.py, migrations/*.sql
src/command/	dispatch.py -- confirm-to-amend, actor attribution, authorize()
src/identity/	resolve_principal(), Principal, the known-actor rule
src/governance/	the domain: model inventory, MRC, products, chain, risk policy, documents, and the LaTeX document builders
src/web/	Flask app factory and blueprints (api_*.py), _startup.py, _theme.py, _helpers.py
src/static/	js/ and css/ -- the console (R7). No build step.
src/templates/	index.html
audit/	the coding-rule audit: one module per scan, plus pdf.py
scripts/	pg.sh, start.sh, dev_app.sh, restore_scratch.sh, backup_db.sh, and the one-off data migrations
docs/	coding_rules.md, user_guide.md, permissions_design.md, mssql-immutability.md, windows-setup.md, design/, and this guide
tests/	unit, integration, governance, web, audit, e2e, journey

© 2022–2026 MKM Research Labs. The software is licensed under the MIT License; see LICENSE.